Neural Network Performance Analysis on Regression and Classification Problem

Paula Solange Gund, Elia Sundström, Raji Nabih Youssef
(Dated: November 9, 2026)

In recent years, neural networks have gained significant attention due to their wide range of applications, from medical diagnosis to economics and physics . In this project, we investigate the performance of Feed-forward neural networks trained using backpropagation and compare them with classical methods on two different tasks: regression with noisy Runge-function data and classification of handwritten digits from MNIST. For regression, we compared gradient descent, RMSprop, and Adam, and studied network architecture, activation functions, and L_1 and L_2 regularization. On the fixed 20-point test set, the lowest test MSE was (0.0083) for a ReLU network with two hidden layers of 100 nodes each, the degree-8 OLS model had a test MSE of (0.0187). In cross-validation, the tested neural network settings gave MSE from0.0166 to 0.0176 compared with 0.0193 to 0.0211 for OLS, Ridge, and Lasso. The best penalty settings varied between five-fold and ten-fold cross-validation. For MNIST, the best performing neural network had one hidden layer of 128 ReLU nodes achieved a test accuracy of 96.8%, compared with 92.5% for logistic regression. The confusion matrices provided further insight into digit misclassifications and showed that the neural networks generally performed better than the classical baselines in these experiments.

CONTENTS

Abstract 1

I. Introduction 1

II. Methods 2
A. Neural networks, feed forward and backpropagation 2
B. Network setup for regression problem 3
1. Activation functions 3
2. The cost function 3
3. Procedure for choosing best network settings 4
C. Network Setup for Multiclass Classification Problem 4
1. The Cost Function 4
2. The Accuracy and Confusion Matrix 4
3. The Dataset 5

III. Results and Discussion 5
A. Regression on Runge's function 5
1. MSE cost function and OLS 5
2. Fixing architecture and activation function 5
3. Cross validation with L_1 cost function 8
4. Cross validation with L_2 cost function 9
5. Comparison of all regression models 9
B. Neural Network Classification Analysis 10
1. Hyperparameter Determination 10
2. Model Performance 11

IV. Conclusion 11

A. Use of AI/LLM Tools 12
A.1 AI Tools used 12
A.2 Text writing and editing 12
A.3 Code generation and assistance 12

References 13

B. Code setup 13

I. INTRODUCTION

Regression and classification are two common problems in machine learning. Regression predicts continuous values, while classification assigns observations to different classes. Both can use *feed-forward neural networks* (FFNNs), which adjust their weights and biases during training. Unlike polynomial regression, which uses predefined basis functions, neural networks adjust their weights and biases to model the relationship between inputs and outputs. Their performance depends on network structure, training methods and parameters, and they do not always lead to better predictions.

In the following report, a FFNN with backpropagation was implemented and applied on both a regression and a classification problem. The regression problem builds on the work that was previously conducted in [1], where data generated from Runge's function (1) was analyzed using standard methods such as OLS, Ridge and Lasso regression in various versions. For further details on past analyses, see [1].

$$f(x) = \frac{1}{1+25x^2} \tag{1}$$

In a first step, a FFNN was trained using backpropagation with a simple *mean square error* (MSE) as cost function and the sigmoid function as the activation function for the hidden layers. Networks with different numbers of hidden layers and nodes were tested. *Gradient descent* (GD), *stochastic gradient descent* (SGD), RMSprop, and Adam were investigated using different learning rates γ. The best model was selected, and its results were com-

pared with those obtained using ordinary least squares (OLS) in [1].

Further, in order to obtain an idea of how the activation function impacts the results, Sigmoid, ReLU and a Leaky ReLU were tested as activation functions for the hidden layers. As a final step, L_1 and L_2 regularization were added to the MSE cost function, adding a further hyperparameter λ whose impact was studied using *cross validation* (CV). The results were compared to the L_1 (Lasso regression) and L_2 (Ridge regression) results from [1].

In a final discussion, all the models used for the regression problem are compared and a final verdict is given.

In a classification analysis, a completely separate dataset of handwritten digits was used from [2]. This dataset of 70 000 images of 28×28 pixels was split into the canonical split of $6/7 \approx 85.71\%$ training data and $1/7 \approx 14.29\%$ test data and a neural network was trained on the training data only. The test data was then run through the final model and the accuracy assessed for various values of the hyperparameter (sometimes called *regularization parameter*) λ, learning rate γ and dimensions of the neural network.

The classification results were compared to similar results obtained from *logistic regression* (LR), to see if adding hidden layers improved prediction accuracy enough to justify the extra computational cost. Moreover, a confusion matrix was also used to check which handwritten digits were most misclassified.

Finally, all the different approaches to the classification analyses were compared in their advantages and disadvantages.

In the following report, the theoretical concepts and methods used are discussed in Section II. The results for both the regression problem and the classification analysis are then presented and discussed in Section III. Finally, the conclusion is made in Section IV. Each Section is separated into two subsections, one containing the regression problem, the other containing the classification analysis. All developed code referenced in this report is available via a GitHub repository [1].

II. METHODS

Table I includes a list of the definition of used variables.

[1] https://github.com/elias98-ux/Machine-Learning---Project-2.git

TABLE I: Global terminology and notation used in this report. Additional quantities may be defined in the text.

Quantity	Sign
L_1/ L_2 hyperparameter	λ_{L_1} / λ_{L_2}
Learning rate	γ
Activation function	f
Cost function	C
prediction/ final activation	a_i^L
s	y

“‘latex

A. Neural networks, feed forward and backpropagation

The neural networks used in this report build on two concepts: a *feed-forward pass* to generate predictions, and training of the network parameters using *backpropagation*. The network consists of a layer of M_0 inputs x_i, defined by the data. In a feed-forward pass, the inputs are processed by first calculating the *pre-activations* z_i, then applying an *activation function* $f(z_i) \equiv a_i$ to obtain the *activations* a_i, which serve as inputs for the next layer.

In general, the pre-activations of layer l are calculated from its weights w_{ij}^l and biases b_j^l by the linear combination [3]

$$z_i^l = \sum_{j=0}^{M_{l-1}} w_{ij}^l \cdot a_j^{l-1} + b_j^l \tag{2}$$

The weights and biases are initialized randomly and trained, while M_l denotes the number of *nodes* in layer l. The activations are then calculated by applying a non-linear activation function:

$$a_i^l = f(z_i^l) \tag{3}$$

The activation function f can be chosen for every layer, but is conventionally the same across *hidden layers*, with a different function for the last layer. Activation functions are not subject to training, but changing them is part of tuning a network for the best possible results. More on the activation functions used in this project is described in Section II B 1.

Passing an input through the entire network is one feed-forward pass and returns an *output layer* L with prediction values a_i^L. These are compared to target values y using a *cost function* $C(a, y)$, yielding a measure of the network's accuracy. The choice of cost function is problem specific and the choices made in this project are specified in Sections II B and II C.

The blame for producing a cost is then propagated back to each weight and bias by simply exploiting that by the chain rule[3]

$$\frac{dC}{da_l} = \frac{\partial C}{\partial z^{l+1}} \frac{\partial z^{l+1}}{\partial a^l} \tag{4}$$

Equation (4) relates layer $l+1$ to the previous layer l and allows us to propagate the gradient backwards through the network, giving the procedure the name *backpropagation*. From there, we can calculate the derivatives of the previous layers following the general pattern

$$\begin{aligned} \frac{dC}{dz^l} &= \frac{\partial f}{\partial z^l} \frac{\partial C}{\partial a^l} \\ \frac{dC}{dw^l} &= \frac{\partial C}{\partial z^l} a^{l-1} \\ \frac{dC}{db^l} &= \frac{\partial C}{\partial z^l} \cdot 1 \end{aligned} \tag{5}$$

The gradients dC/dw^l and dC/db^l are the quantities we are interested in as they give us the cost function C's dependence on the weights w_{ij}^l and biases b_i^l. Calculation of these gradients then allows us to *train the network* by updating the weights and biases using any updating scheme of our choice, such as e.g. plain gradient descent.

More details on the implementation of the network setup can be found in Table XII in the Appendix B.

B. Network setup for regression problem

For the regression experiments, the observations were generated from the one dimensional Runge function (1) with Gaussian noise. The function `prepare_runge_data` used `numpy.random.default_rng(2026)` to generate and sort 100 uniformly distributed input values before adding the noise. The data were divided into 80 training and 20 test observations using `train_test_split` with `random_state=2026`. Each network input was a single value of x. Separate `StandardScaler` objects were fitted to the training inputs and targets, and the same scaling was then applied to the test data. Regression errors were calculated in the original target units. For learning over epochs plots, an MSE obtained from standardized targets was multiplied by s_y^2.

1. *Activation functions*

The activation functions considered in the regression problem were sigmoid, ReLU (Rectified Linear Unit) and leaky ReLU. The sigmoid function is defined by

$$\sigma(z) = \frac{1}{1+\exp(-z)}, \tag{6}$$

with the derivative

$$\sigma'(z) = \sigma(z)[1-\sigma(z)], \tag{7}$$

and was for a long time the standard choice for the activation function. However, for large values of $|z|$, its derivative becomes smaller and thus slows down learning [3].

The ReLU activation [3] fixes the problem of the decreasing learning rate. It is defined by

$$\mathrm{ReLU}(z) = \begin{cases} z, & z > 0 \\ 0, & z \leq 0, \end{cases} \tag{8}$$

and its derivative by

$$\mathrm{ReLU}'(z) = \begin{cases} 1, & z > 0 \\ 0, & z < 0. \end{cases} \tag{9}$$

However, ReLU introduces a new flaw: if the updated weights force the input to be zero, it leads to a zero output. In this case, the gradient also becomes zero, with the consequence that the unit remains zero for the remainder of training, potentially leading to significant errors. There are various fixes for this issue. This report mainly considers leaky ReLU, in which the negative values of z are instead suppressed by a small constant α:

$$\mathrm{ReLU}_{\mathrm{leaky}}(z) = \begin{cases} z, & z > 0 \\ \alpha z, & z \leq 0 \end{cases}, \qquad \alpha = 0.01 \tag{10}$$

whereby the outputs and derivatives remain non-zero for negative inputs [3].

At $z = 0$, the code used a derivative of 0 for ReLU and α for Leaky ReLU. These are implementation choices, since neither function has a classical derivative at zero.

2. *The cost function*

For the network of the regression problem, we are using the cost function (11)

$$\begin{aligned} C(a,y) = &\underbrace{\frac{1}{n}\sum_{i=0}^{M_L}(a_i^L - y_i)^2}_{\equiv C_{MSE}(a,y)} \\ &+ \lambda_{L_1} \underbrace{\sum_{l=0}^{L}\sum_{i,j=0}^{M_l,M_{l-1}} |w_{ij}^l|}_{\equiv C_{L_1}} \\ &+ \lambda_{L_2} \underbrace{\sum_{l=0}^{L}\sum_{i,j=0}^{M_l,M_{l-1}} (w_{ij}^l)^2}_{\equiv C_{L_2}} \end{aligned} \tag{11}$$

which has the derivative

$$\begin{aligned}\frac{\partial C}{\partial w_{ij}^l} &= \frac{2}{n}\sum_{k=1}^{n}(a_k^L - y_k)\frac{\partial a_k^L}{\partial w_{ij}^l} \\ &+ \lambda_{L_1} s_{ij}^l \\ &+ 2\lambda_{L_2} w_{ij}^l\end{aligned} \tag{12}$$

where $s_{ij}^l = \text{sign}(w_{ij}^l)$ for $w_{ij}^l \neq 0$ and $s_{ij}^l = 0$ for $w_{ij}^l = 0$. The values for the penalty hyperparameters λ_{L_1} and λ_{L_2} are to be defined and can also be chosen to vanish, reducing the cost function to pure mean least square (MSE).

3. Procedure for choosing best network settings

In our treatment of the regression problem, the following procedure was performed to fix the optimizer, activation function, architecture, learning rate γ, and penalty hyperparameters λ_{L_1} and λ_{L_2}.

First, `create_layers_batch` initialized the weights using independent normal random values and the biases to a constant $b_j^l = 0.01$. The same initialization was used for all experiments, while `optimiser_step` was used for parameter updates.

Then, using only the MSE cost function C_{MSE} from (11), the network was trained using plain gradient descent and stochastic gradient descent with RMSprop and Adam, investigating different learning rates γ. This was tested for two architectures: `(50,)` and `(100,100)`.

Each regression ran for 1000 epochs. Plain gradient descent used the full training set for each update, while RMSprop and Adam used shuffled mini-batches of up to 32 observations. Training MSE was recorded every 20 epochs.

Using the optimizer with the smallest validation MSE, the three activation functions (6), (8) and (10) were tested for four architectures: `(50,)`, `(50, 50)`, `(100,)`, `(100, 100)`. The combinations were compared using training and test MSE, and the best was chosen to proceed.

Finally, 5- and 10-fold cross-validation was performed over the learning rate γ and penalty hyperparameters λ_{L_1} and λ_{L_2} to determine their optimal values. The NN results were then compared to OLS, Ridge and Lasso results from [1].

C. Network Setup for Multiclass Classification Problem

For classification problem the responses can only be from a discrete set of classes $k = 0, 1, ..., K-1$. The problem is to predict the correct class for the design matrix $\boldsymbol{X} \in \mathbb{R}^{n\times p}$, where n is the number of samples with p features.

1. The Cost Function

The cost function for classification problems is the cross-entropy. For a binary target $y_i \in \{0, 1\}$ and a sigmoid probability $p_i = \sigma(z_i)$, this loss and its derivatives are

$$C_{\text{BCE}} = -\frac{1}{n}\sum_i \left[y_i \log p_i + (1-y_i)\log(1-p_i)\right], \tag{13}$$

$$\frac{\partial C_{\text{BCE}}}{\partial p_i} = \frac{p_i - y_i}{n p_i(1-p_i)}, \qquad \frac{\partial C_{\text{BCE}}}{\partial z_i} = \frac{p_i - y_i}{n}. \tag{14}$$

For the multiclass case, let Z_{ik} be the output score for observation i and class k, and let Y_{ik} be its one-hot target. The softmax probabilities, cross-entropy loss and gradient with respect to the output scores are

$$P_{ik} = \frac{e^{Z_{ik}}}{\sum_{r=1}^{K} e^{Z_{ir}}}, \tag{15}$$

$$C_{\text{CE}} = -\frac{1}{n}\sum_{i,k} Y_{ik}\log P_{ik}, \tag{16}$$

$$\frac{\partial C_{\text{CE}}}{\partial Z_{ik}} = \frac{P_{ik} - Y_{ik}}{n}. \tag{17}$$

The function `softmax` subtracted the largest score in each row before exponentiation, leaving the probabilities unchanged while avoiding very large exponentials. The gradient is then passed backward through the network using the layer-by-layer procedure described earlier.

The choice of activation function for the hidden layers is free, while the output layer's activation function is fixed by the range of the target. For classification, the output layer uses the softmax function, Eq. (15), which gives predicted probabilities between 0 and 1 for each class, normalized to sum to one.

2. The Accuracy and Confusion Matrix

The accuracy of the results of a classification problem is measured by the fraction of correctly classified samples out of all classifications

$$\text{accuracy} = \frac{\text{TP} + \text{TN}}{\text{TP} + \text{TN} + \text{FP} + \text{FN}} = \frac{1}{n}\sum_{i=1}^{n} I(y_i = \tilde{y}_i) \tag{18}$$

where I is the indicator function; 1 if $y_i = \tilde{y}_i$, otherwise 0.

The confusion matrix visualizes the results of a classification problem. For binary classification, it arranges the true positives (TP), true negatives (TN), false positives (FP), and false negatives (FN) into a 2×2 matrix, as shown in Table II. For N classes, it expands to an $N \times N$ matrix, where the diagonal entries represent correctly classified samples and the off-diagonal entries represent misclassifications [3].

	Predict 1	Predict 0
True 1	TP	FN
True 0	FP	TN

TABLE II: The confusion matrix for two classes

3. *The Dataset*

The dataset used for classification is the full MNIST dataset [2], consisting of 70 000 images of handwritten digits between 0 and 9. Each image is represented by a 28×28 pixel grid.

The MNIST dataset is divided into a training set 60 000 images a test set with 10 000 of the images ($\approx 14.3\%$). The training set was subsequently divided into a validation set containing 25% of the 60 000, images with the remaining 75% used for training. The final division of data is approximately 14.3%, 21.4%, and 64.3% for the test, validation, and training sets respectively. The validation set is used to tune the hyper parameters of the training of the neural network [3].

III. RESULTS AND DISCUSSION

A. Regression on Runge's function

1. *MSE cost function and OLS*

In a first step, Figure 1 compares different optimizers for different learning rates. Gradient descent (GD), RMSprop, and Adam for the one-hidden-layer network with 50 nodes and the two hidden layer network with 100 nodes for each layer. This comparison shows which settings reached the lowest training error within the 1000-epoch. From 1 we can see that the learning rate improved the training error reached within the fixed epochs number. For the 50-node sigmoid network, increasing the **GD** learning rate from 0.001 to 0.1 reduced training MSE remarkably from 0.092899 to 0.018031. The two-layer network showed a similar change, from 0.093668 to 0.018659, but did not achieve lower training MSE than the one layer network.

RMSprop and Adam achieved lower training MSEs than GD. The effect of increasing the learning rate was not the same for both optimizers. For Adam, the lowest MSE among the three tested rates was achieved at $\gamma = 0.03$ for both network architectures. RMSprop achieved its lowest MSE for the two-layer network at $\gamma = 0.003$. Increasing the rate to $\gamma = 0.03$ slightly increased the MSE for this network.

These results are related to how the optimizers update the parameters. RMSprop scales each coordinate using a running average of squared gradients. Adam uses this scaling and also averages the gradient direction.

Both architectures selected Adam at $\gamma = 0.03$ on training MSE. Table III gives their epochs and test errors. Both networks had lower test MSE than the degree-eight OLS, and the two-layer sigmoid network had the lowest test MSE of the completed experiments.

Model	Optimizer	γ	Epoch	Train MSE	Test MSE
OLS, degree 8	—	—	—	—	0.018712
NN (50)	Adam	0.03	880	0.013818	0.013511
NN (100, 100)	Adam	0.03	980	0.012217	0.009002

TABLE III: Part B: selected sigmoid networks and OLS evaluated on the same 20 test observations. Network selection used training MSE.

Figure **??** shows how each model fits the data. The OLS curve stays below the Runge function at its peak and oscillates near the ends of the interval. Both neural network curves reach closer to the actual curve. Among the two NN networks, the one-layer network shows better fit to the actual curve than the two-layer network. A lower test MSE means that the model fits the exact function within the given interval but does not necessarily mean that a it will fit it everywhere.

Given that Adam returned the best results overall in this section, all further analyses were performed using only Adam as optimizer from here on.

2. *Fixing architecture and activation function*

We examined how the activation function and network size affected the fit to the Runge function. We tested sigmoid, ReLU, and Leaky ReLU with one or two hidden layers and 50 or 100 nodes in each layer. Adam's learning rate was fixed at $\gamma = 0.03$ for this part. It is later tuned in more detail using CV (Sections III A 3 and III A 4). For each network, we kept the epoch with the lowest training MSE and calculated the test MSE at that epoch.

TABLE IV: Training and test MSE for several network configurations at the epoch with the lowest training MSE. All runs used Adam with $\gamma = 0.03$, a maximum of 1000 epochs, batches of 32, and seed 2026.

Activation	Hidden widths	Epoch	Training MSE	Test MSE
Sigmoid	(50)	880	0.013818	0.013511
Sigmoid	(100)	940	0.013748	0.014095
Sigmoid	(50, 50)	940	0.012654	0.011320
Sigmoid	(100, 100)	980	0.012217	0.009002
ReLU	(50)	500	0.013789	0.014234
ReLU	(100)	980	0.013659	0.014909
ReLU	(50, 50)	1000	0.012019	0.011012
ReLU	(100, 100)	760	0.012577	0.008294
Leaky ReLU	(50)	500	0.013803	0.014720
Leaky ReLU	(100)	980	0.013684	0.014951
Leaky ReLU	(50, 50)	880	0.012209	0.011520
Leaky ReLU	(100, 100)	940	0.011939	0.009306

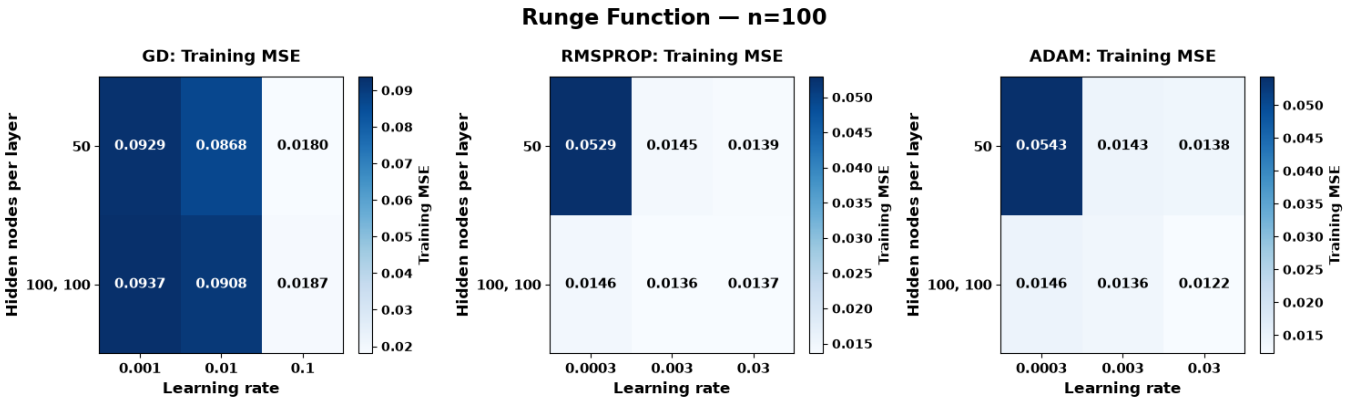


FIG. 1: Training MSE for the two sigmoid network architectures across the tested learning rates for GD, RMSprop, and Adam.

Table IV shows the results.

For one hidden layer, sigmoid had the lowest test MSE at both `(50,)` and `(100,)`. Increasing the number of nodes from 50 to 100 increased it test MSE while it decreased its training MSE. Adding a second layer for sigmoid lowered both training MSE and test MSE. Sigmoid reached its lowest training and test MSE for the configuration `(100,100)`.

The ReLU and Leaky ReLU showed a similar difference between one and two hidden layers. Adding a second layer lowered test MSE for ReLU, and for Leaky ReLU. For both activations, changing number of nodes for the one-layer network slightly lowered training MSE but raised test MSE. In the two-layer networks, increasing the number of nodes from 50 to 100 nodes lowered test MSE for both activations but the training MSE increased for ReLU. Both activations reached their best **test MSE** for the architecture `(100,100)`, while best training MSE was reached for ReLU in `(50,50)` architecture and for Leaky ReLU in `(100,100)`.

The training MSE across the activation functions and network architectures is plotted in Figure 3. From it we observe that the three activations showed the same training MSE for the same one-layer network. The two-layer networks generally had lower training MSE than the one-layer networks. The exception was ReLU, where the `(50, 50)` network had a lower training MSE than the `(100, 100)` network.

Also, test MSE was studied for the same network configurations as shown in Figure 4. From it , we can observe that adding a second hidden layer decreased test MSE at both number of nodes 50 and 100 for all three activations. The lowest MSE value was registered for the two-layer ReLU network with 100 nodes. We can also see that increasing the number of nodes in the one-layer networks raised test MSE for all three activations.

The training and test MSE figures give different rankings. Leaky ReLU with two layers of 100 nodes had the lowest training MSE, while ReLU with two layers of 100 nodes had the lowest test MSE.

For each activation function, we selected the network configuration that gave the lowest training MSE and plotted how its training MSE changed over the epochs. Figure 5 shows the results for sigmoid with architecture of `(100,100)`, ReLU with `(50,50)`, and Leaky ReLU with `(100,100)` nodes.

The sigmoid training MSE dropped sharply at the start and then it staid almost flat. The ReLU and Leaky ReLU curves decreased more gradually and fluctuated during training. The dotted vertical lines mark the epochs with the lowest training MSE: 980, 1000, and 940 for Sigmoid, ReLU and Leaky ReLU respectively, Table IV.

Finally, the test MSE was compared with training MSE for the same architectures and plotted against each other as shown in Figure 6 Sigmoid (100), ReLU (50) and (100), and Leaky ReLU (50) and (100) one-layer networks showed higher test MSE than training MSE. The other seven configurations had lower test MSE than training MSE. Higher test MSE can be a sign of overfitting, where the model fits the training data better than unseen data. At the same time, lower test MSE does not mean that overfitting is absent, especially when the test set is small. Our experiment used one split with 20 test points. These results show what happened in this split, but they are not enough to say that one activation function is generally better or that wider networks always overfit.

The results in Fig. 6 show the smallest test MSE for the ReLU activation function and a `(100, 100)` architecture. However, there the gap between training and test error is quite large, hinting towards overfitting. Thus, from here on, we will proceed with the ReLU activation function at an architecture of `(50, 50)`, as this activation function has the smallest test and training MSE at this architecture.

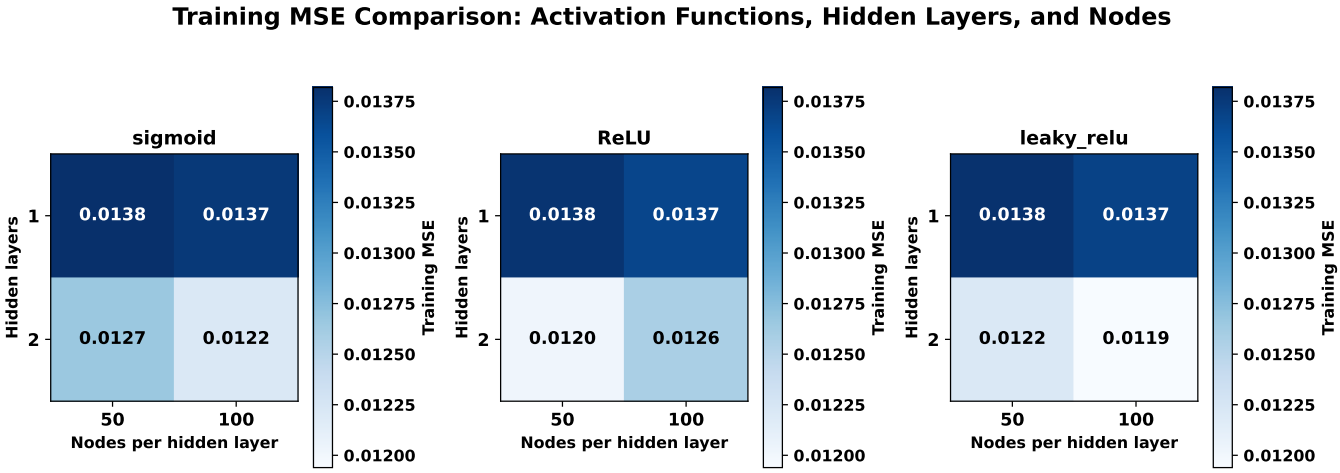


FIG. 3: Training MSE for activation function in different architectures.

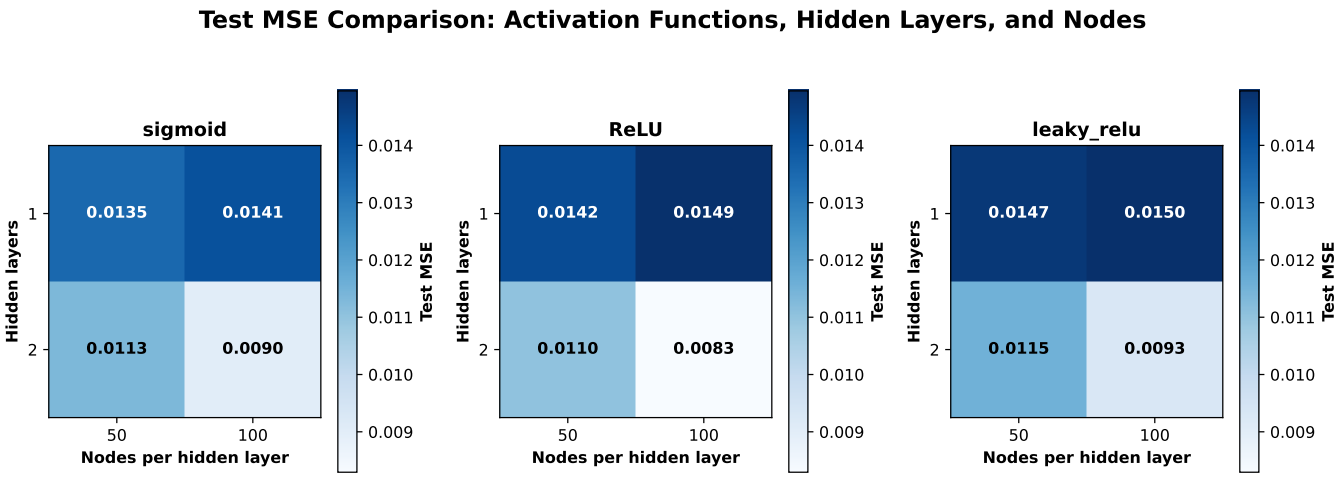


FIG. 4: Test MSE for activation function in different architectures.

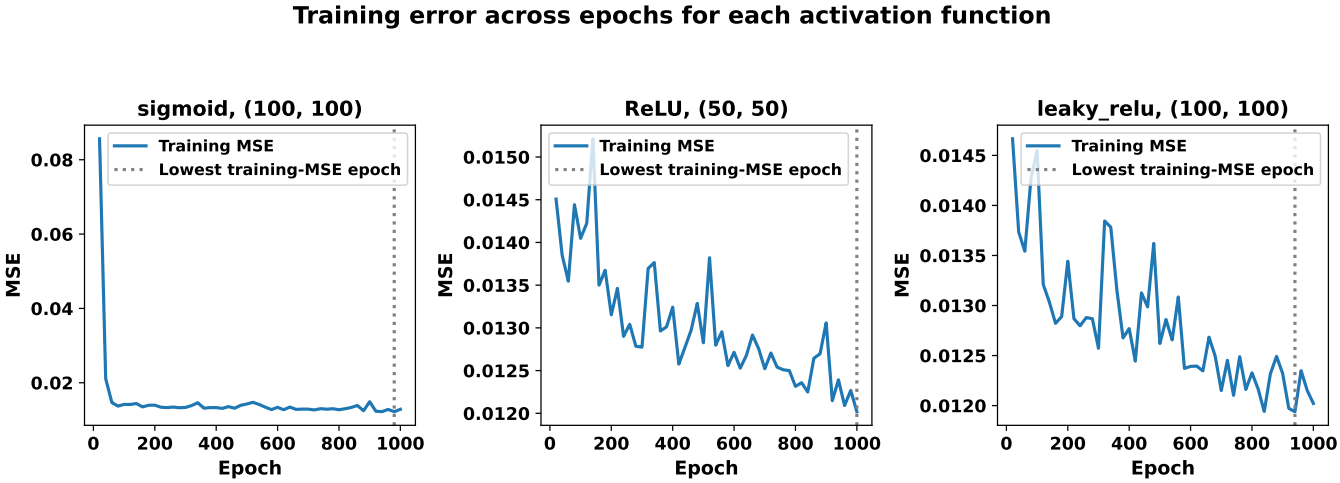


FIG. 5: Training MSE over epochs for the configuration with the lowest training MSE for each activation.

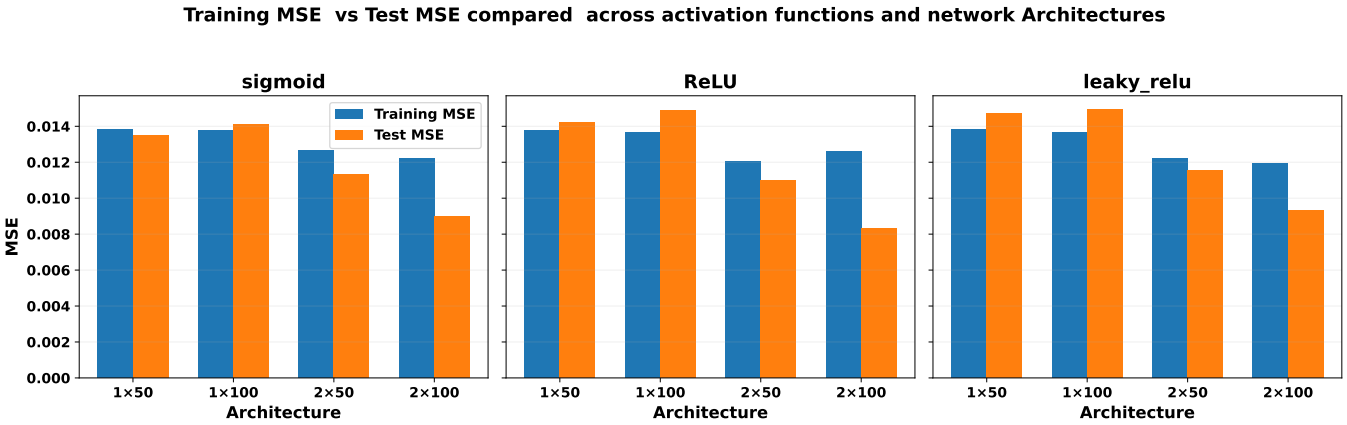


FIG. 6: Training MSE over epochs for the configuration with the lowest training MSE for each activation.

3. *Cross validation with L_1 cost function*

Having fixed the optimizer, architecture and activation function, cross validation is now used to fix the learning rate γ and the L_1 penalty hyperparameter λ_{L_1}.

Optimizer	Adam
Architecture	`(50,50)`
Activation function	ReLU
Epochs	400
γ sweep	$\gamma \in$ {`logspace(-2.6,-0.5)`}
λ_{L_1} or λ_{L_2} sweep	$\lambda_{L_1}, \lambda_{L_2} \in \{0,$ `logspace(-3,-1)`$\}$

TABLE V: Network setup for the CV runs using non-zero L_1 or L_2 penalty hyper parameters λ_{L_1} or λ_{L_2}.

Note that the cross validation sweep over λ_{L_1} also includes the case where $\lambda_{L_1} = 0$. This was purposely included to provide an easy comparison of the results to the pure MSE case where no penalty parameter is used. The sweep range for γ was defined to expand beyond the previously fixed and tested $\gamma = 0.03$ in both positive and negative directions.

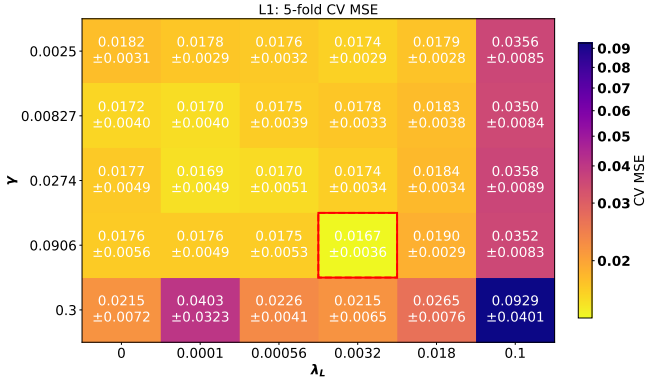


FIG. 7: CV MSE and standard deviations of 5-fold cross validation in γ and λ_{L_1} using the setup from Table V

In Figure 7, the best result is pointed out by a red border around the cell that corresponds to the combination of learning rate and penalty parameter that yielded the smallest cross-validated MSE (CV MSE). This is obtained in 5-fold cross validation for the combination $\gamma = 0.0906$ and $\lambda_{L_1} = 0.0032$, resulting in a CV MSE of 0.0167 with a standard deviation between the folds of 0.0036, which lies somewhere in the middle of our sweeping ranges for γ and λ_{L_1}. Compared to the results that use $\lambda_{L_1} = 0$, meaning that the model reduces to using the plain MSE cost function, adding a non-zero penalty parameter improves the results, though not extremely.

The results of the 10-fold CV are shown in Figure 8, where the least overall result CV MSE is reached for the combination $\gamma = 0.0274$ and $\lambda_{L_1} = 0.00056$. This combination yields a CV MSE and standard deviation of 0.0162 ± 0.0043. Again, the best result obtained with a non-zero penalty are better than the ones where $\lambda_{L_1} = 0$.

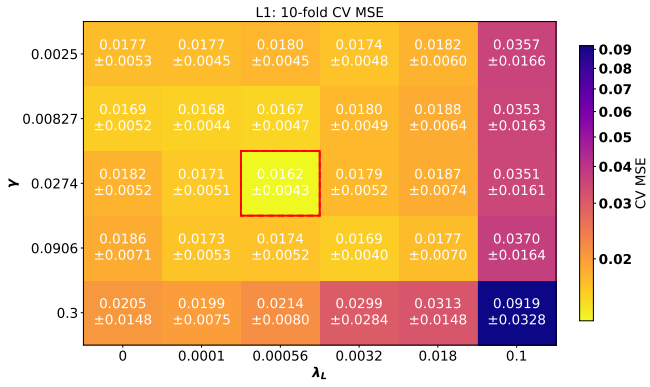


FIG. 8: CV MSE and standard deviations of 10-fold cross validation in γ and λ_{L_1} using the setup from Table V

Compared between each other, the result of the 5-fold CV (Fig. 7) comes very close to the accuracy of the 10-fold CV (Fig. 8) with the CV MSE of all best results falling into the error margins of each other. We also ob-

serve that performance generally becomes poor for larger values in λ_{L_1}, and both very large and very small values of the learning rate γ. In both 5- and 10-fold CV, introducing a small penalty parameter decreases the CV MSE in comparison to the plain MSE cost function, independent of the learning rate.

4. *Cross validation with L_2 cost function*

The analogous analysis as in the previous section also leads to similar results when introducing a non-zero L_2 hyperparameter in the cost function (11). In Figure 9, the combination of learning rate γ and penalty parameter λ_{L_2} that yield the smallest CV MSE and its standard deviation is $\gamma = 0.0274$ and $\lambda_{L_2} = 10^{-4}$, yielding 0.0162 ± 0.0047. We again see very poor performance of the models where λ_{L_2} is large and using a cost function with an L_2 penalty clearly defeats using a pure MSE cost function. It is, however, important to note that the selected λ_{L_2}-value here is the smallest non-zero value that was actually tested. It would be very interesting to extend the sweeping range to even smaller (but non-zero) values of λ_{L_2}.

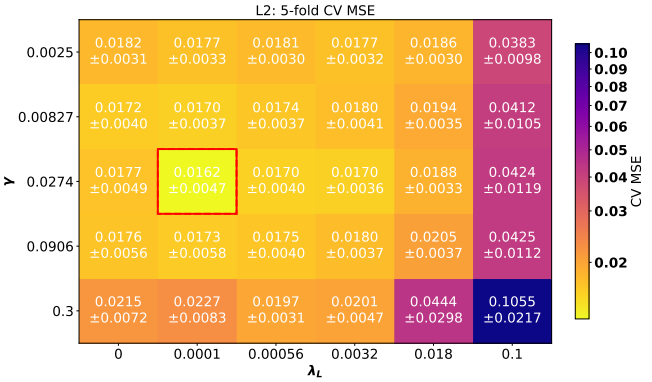


FIG. 9: CV MSE and standard deviations of 5-fold cross validation in γ and λ_{L_2} using the setup from Table V

Figure 10 shows the results for the 10-fold CV, where the $\lambda_{L_2} = 10^{-4}$ that produces the lowest CV MSE is the same as in the 5-fold CV, but the chose learning rate $\gamma = 0.00827$ is a bit smaller. Note, however, that the same CV MSE was obtained by the bigger values for both γ and λ_{L_2} on the diagonal of the heatmap, with only the standard deviation between folds a bit larger. Again, it would be interesting to test if even smaller values of λ_{L_2} could further improve the CV MSE, as the chosen value is the smallest one we tested.

Between 5- and 10-fold CV, we again observe that the 10-fold CV returns a slightly smaller CV MSE for the optimal combination of γ and λ_{L_2}, though they are within each others error margin. The 10-fold CV also favours a smaller learning rate, though it generally obtains slightly better results than 5-fold CV even for combinations of γ and λ_{L_2} that are not optimal. This could be due to a

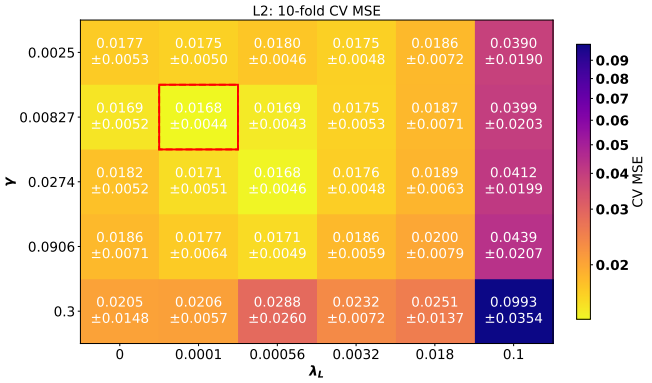


FIG. 10: CV MSE and standard deviations of 10-fold cross validation in γ and λ_{L_2} using the setup from Table V

reduced variation between folds due to the fact that the training dataset of 90 data points is slighly bigger than that of 80 data points in 5-fold CV.

5. *Comparison of all regression models*

After all of the above analyses, we present an overview of the results of our regression analysis in Table VI, including the cross validation results from [1], using the optimal polynomial degrees for each method.

Model	CV folds	γ	λ_{L_1}	λ_{L_2}	CV MSE ± STD
OLS, deg 11	5	–	–	–	0.0211
NN (50,50)	5	0.00827	–	–	0.0172 ±0.0040
Ridge, deg 12	5	–	–	$1.7981 \cdot 10^{-7}$	0.0208
NN (50,50)	5	0.0274	–	10^{-4}	0.0162 ±0.0047
Lasso, deg 11	5	–	10^{-6}	–	0.0211
NN (50,50)	5	0.0906	0.0032	–	0.0167 ±0.0036
OLS, deg 11	10	–	–	–	0.0202
NN (50,50)	10	0.00827	–	–	0.0169 ±0.0052
Ridge, deg 12	10	–	–	$1.5983 \cdot 10^{-7}$	0.0193
NN (50,50)	10	0.00827	–	10^{-4}	0.0168 ±0.0044
Lasso, deg 13	10	–	$1.4385 \cdot 10^{-5}$	–	0.0201
NN (50,50)	10	0.0274	0.00056	–	0.0162 ±0.0043

TABLE VI: Final comparison of different models in regression problem. Detailed setup of the NN is described in Table V. OLS, Ridge and Lasso results from [1].

Here, we can see how our neural network performs in comparison to standard regression methods. In the last columns of Table VI, we can clearly see that the CV MSE

Activation	Hidden units	Learning rate	CV accuracy
sigmoid	128	0.010	0.9506 ± 0.0022
ReLU	128	0.010	0.9507 ± 0.0039
leaky ReLU	128	0.010	0.9539 ± 0.0027

TABLE VII: Results for cross-validation for different activation functions (sigmoid, ReLu, and leaky ReLu). The cross validation checked the accuracy for different learning rates and layer architecture, see FIG. 11 for results

of a `(50, 50)` NN defeats a regular regression model from [1] in all cases, though sometimes only very closely when considering the standard deviation between folds.

The penalties λ yielding the best results differ quite significantly in orders of magnitude between the results of [1] and the NN, with the Ridge/ Lasso penalties of orders 10^{-6} and 10^{-7}, though in the L_2 penalized networks, it is not clear whether penalties $\lambda_{L_2} < 10^{-4}$ may have produced even better results. As already for the regular regression models, L_2 or Ridge returns the smallest CV MSE in both 5 and 10-fold CV.

B. Neural Network Classification Analysis

In this part of the report, the use of a neural network for the classification of handwritten numbers is studied. The results are compared with classification using logistic regression, i.e., a neural network with zero hidden layers.

1. *Hyperparameter Determination*

The learning rate, the number hidden layers, and the number of nodes in the hidden layer was decided through use of 5-fold cross-validation; the combination with the highest cross-validation accuracy was with a single layer with 128 nodes, the learning rate 0.01 (see FIG. 11 for ReLU architecture results) for the three tested activation functions, see table VII. All activation functions had similar CV accuracies, with leaky ReLU achieving a marginally higher accuracy. However, given the overlap of their error estimates, see FIG. 12, there is no clear evidence of significant differences in performance between the different activations. The decision was therefore made to use ReLU for the rest of the analysis due to it giving slightly higher accuracies when training the neural network. The cross-entropy was used as the cost function and the softmax function as the activation function of the output layer.

Adam was chosen as the optimizer due to it having the best accuracy and least indication of overfitting, see table IX. Gradient decent achieved substantially lower accuracy and was therefore discarded. RMSprop achieved a test accuracy only slightly lower than Adam's, but showing a greater gap between training and test error, 2.5

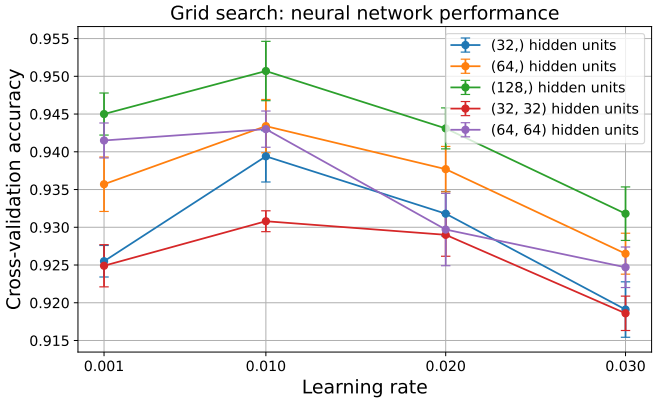


FIG. 11

percentage points, suggesting a greater degree of overfitting.

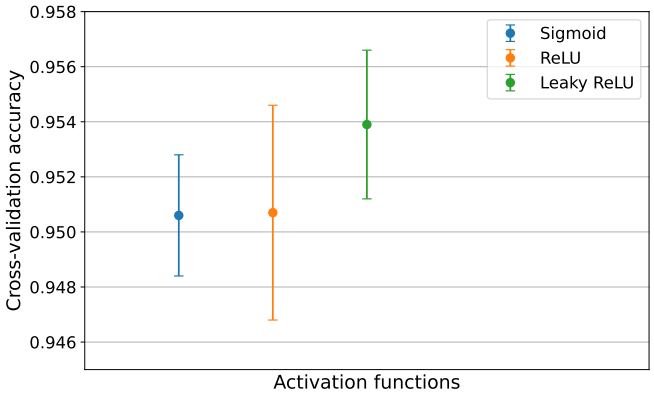


FIG. 12: Comparison of the classification accuracy of the best performing neural network architecture using different activation functions, see table VII.

The regularization parameter $\lambda = 0$ was chosen due to the apparent lack of improvement in the accuracy for non-zero values of λ, see table VIII. One possible explanation for the reduced accuracy is that the regularization penalizes large weights, potentially restricting the model's ability to capture complexities of the MNIST dataset.

λ	Test accuracy
0	0.9683
0.00001	0.9658
0.0001	0.9679
0.001	0.9487
0.01	0.9070

TABLE VIII: Results when varying the regularization parameter λ for a neural network with one hidden layer for a classification problem. The MNIST dataset of digits between 0 and 9 was used.

2. Model Performance

Logistic regression achieved a training and test accuracy of 94% and 92.5%, respectively, with a gap of 1.5 percentage points. The neural network with Adam had the training and test accuracies 98.7% and 96.8%, resulting in larger gap of approximately 1.9 percentage points. These relatively small differences suggest both models exhibit only limited overfitting. See table IX for a overview of the results. The test accuracy increased by about 4.3 percentage points when using the neural network. The confusion matrices are shown in FIG. 13 and 14 for the logistic regression and the best classification neural network respectively.

	Training accuracy	Test accuracy
Logistic Regression	0.9399	0.9253
Neural Network (GD)	0.2274	0.2313
Neural Network (RMSprop)	0.9926	0.9674
Neural Network (Adam)	0.9877	0.9683

TABLE IX: Results for a neural network using ReLU as the activation function in one hidden layer and logistic regression for the classification problem with the MNIST dataset of digits between 0 and 9

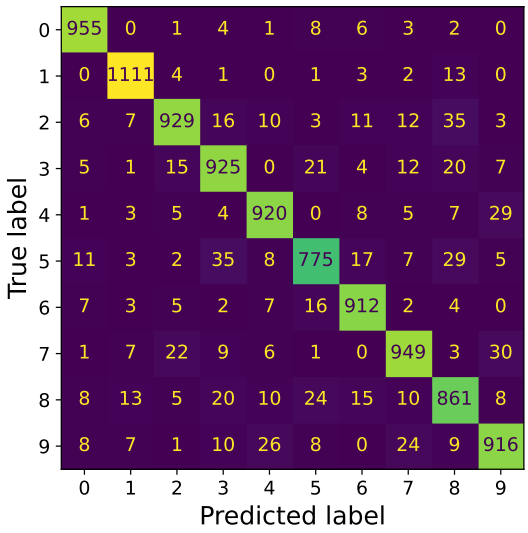


FIG. 13: The confusion matrices for the classification of 10 digits for $0-9$ for Logistic regression

The confusion matrix is dominated by the diagonal elements for both models, indicating that most digits are. For the logistic regression, the majority of the larger off-diagonal entries occur in approximately symmetric pairs, e.g, $3 \leftrightarrow 2$ has 15 instances of $2 \rightarrow 3$ (confusing a two with a three) and 16 instances of $3 \rightarrow 2$ (confusing a three with a two). Other commonly confused pairs are $4 \leftrightarrow 5$, $4 \leftrightarrow 9$, $7 \leftrightarrow 9$, and $8 \leftrightarrow 5$. However, this approximate symmetry is not consistent for all pairs. For instance,

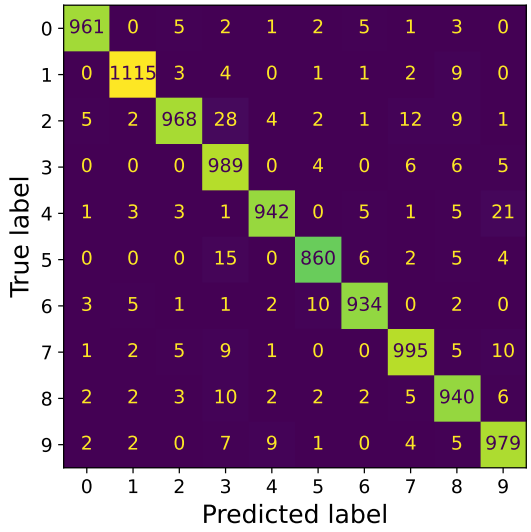


FIG. 14: The confusion matrices for the classification of 10 digits for a neural network with one hidden layer with 128 ReLU nodes and softmax for the output layer.

$2 \rightarrow 8$ occurs 35 times while $8 \rightarrow 2$ occurs only 5 times, see 13.

In contrast, the neural network exhibit less symmetric behavior, with the mislabeling predominately occurring in one direction. Furthermore, the misclassifications are generally concentrated on a fewer pairs. For example $2 \rightarrow 3$ occurs 28 while $3 \rightarrow 2$ does not occur at all, this is also the only significant case where the neural performed worse than the logistic regression. Similarly, $4 \rightarrow 9$ occurs 21 times while $9 \rightarrow 4$ only 9 times. Overall, these results demonstrate that the neural network with one hidden layer performs better than logistic regression, see 14.

The asymmetry of the misclassifications might be caused by the appearance of some handwritten digits not matching the features learned by the model. This effect may be more pronounced when adding the hidden layer due to it being more able to capture complex features through its non-linear activation function. For example, a handwritten version two may share more visual features with a three which the neural network catches than vice versa. Consequently, the neural network with the hidden layer might classify twos to the class of threes without making a comparable error in the opposite direction.

IV. CONCLUSION

After comparing the neural networks with the degree-8 OLS model, both NN `(50,)` and NN `(100,100)` had lower test MSE on the Runge data. The OLS curve underestimated the peak and oscillated near the edges of the interval.

We compared GD, RMSprop, and Adam for both net-

work architectures. Adam gave the lowest training MSE for each architecture. Both selected networks also had lower test MSE than OLS.

Among the tested network configurations, ReLU with two hidden layers of 100 nodes each had the lowest test MSE. Leaky ReLU with the same architecture had the lowest training MSE. The differences between training and test MSE may indicate poorer performance on unseen data for some configurations, but they do not confirm overfitting. A validation set or cross-validation would be needed to assess this during model selection.

The degree-eight OLS model had been selected using test MSE in an earlier study. Its test MSE is therefore not an independent estimate of how that selected polynomial would perform on new data. A fair final comparison would require selecting both models without using the test set and then evaluating them on a separate, unseen test set.

With the chosen setup of a `(50, 50)` NN using Adam as optimizer, ReLU as activation function and 400 epochs, cross validation was performed on both the learning rate γ and the L_1 and L_2 penalty parameters, separately. At their best parameters, the neural networks were able to outperform standard methods from [1] for all three methods (OLS, Ridge, Lasso) in terms of CV MSE, as can be seen in Table VI.

The smallest CV MSE of 0.0162 ± 0.0047 was obtained in the 5-fold CV of the NN with a L_2 penalty parameter of $\lambda_{L_2} = 10^{-4}$ and learning rate $\gamma = 0.0274$. The identical CV MSE with a different standard deviation (0.0162 ± 0.0043) was also achieved in the 10-fold CV of a NN using the same γ, but a L_1 penalty of $\lambda_{L_1} = 0.00056$.

For the classification of the handwritten numerals between 0 and 9, the best results were achieved with a neural network with on hidden layer containing 128 hidden neurons and ReLU as the activation function. The network architecture and learning rate were selected using 5-fold cross-validation, which found 0.01 as the best choice for learning rate. Introducing a regularization parameter did not improve the classification accuracy and therefore it was set to zero.

The final neural network achieved a test accuracy of 96.8%, corresponding to a improvement of 4.3 percentage points compared with logistic regression. An investigation of the confusion matrices (see FIG. 13 and 14) revealed that logistic regression had a roughly symmetric misclassification distribution, whereas the neural network showed more pronounced asymmetries. one potential explanation is that the neural network's nonlinear activation functions gives it a greater ability to learn more complex features, resulting in the asymmetry. Overall, the results show the improved classification performance compared with logistic regression.

Appendix A: Use of AI/LLM Tools

A.1 AI Tools used

AI Tools used:

- GPT-5.6 Sol(September 2026),
- GitHub Copilot (VS Code, October 2026)
- Claude Sonnet 5.5 Browser Version

A.2 Text writing and editing

The levels of AI use for text are defined as follows:

- 0 – **None:** No AI assistance was used.
- 1 – **Language only:** AI assistance was limited to grammar, spelling, and minor phrasing corrections.
- 2 – **Editorial:** The model rewrote or restructured sentences, while the ideas were supplied by the author and the result was checked by the author.
- 3 – **Generative:** The model drafted substantial text based on the author's prompts. The author subsequently revised and verified the text and takes full responsibility for its content.

TABLE X: AI declaration text

Section	Level	Notes
Abstract	2	Was given a written text, then it was checked for mistakes and condensed. It also helped with restructuring some sentences.
Introduction	2	it checked for mistakes and made the given text more coherent
Methods	2	Used as a fact checker and challenger.
Results	2	It was used as a fact checker and challenger of discussions that were already written. Some corrections that came from this were implemented in our own words (and understanding).
Conclusion	2	Gave suggestions for some modifications
Formatting	1	occasional use throughout to format

A.3 Code generation and assistance

The levels of AI use for code assistance are defined as follows:

- 0 – **None:** No AI assistance was used.
- 1 – **Debugging:** The model identified a bug, you fixed it.

- 2 – **Snippet:** A function, loop or short block that you integrated and tested.
- 3 – **Skeleton:** The model generated the structure, you filled in the logic
- 4 – **Substantial:** The model wrote most of the file, you adapted, tested and commented it.

TABLE XI: AI declaration code generation and assistance

Section	Level	Notes
Code, tests, implementation	2	Helped with marked snippets of code, particularly for plotting, and debugging
parts b,c in the notebook except for plotting	2,3	Helped with merging snippet codes from project 1 and weekly exercises . debugging loops and syntax errors
parts b,c in the notebook all plotting code	4	All the plotting code was generated by AI based on what was instructed to do
parts d) - e) in the notebook, except for plotting	1,2	AI used for debugging and filling in correct syntax and adapt the snippets imported from weekly exercises
part d)-e) in the notebook all plotting code	4	Entirely generated, then checked if the correct arrays were plotted and occasional modified

[1] Paula Gund Elia Sundström, Raji Youssef. Project 1. -, 2026.
[2] Yann LeCun, Corinna Cortes, and CJ Burges. Mnist handwritten digit database. *ATT Labs [Online]. Available: http://yann.lecun.com/exdb/mnist*, 2, 2010.
[3] Morten Hjorth-Jensen. *Machine Learning and the Physical Sciences*. Springer, 2026.

Appendix B: Code setup

Function name	Arguments	Role
`create_layers_batch`	Input size, layer sizes, RNG	Initializes the neural network layers and weights.
`feed_forward_batch`	Inputs, layers, activation functions	Computes the network output through forward propagation.
`feed_forward_saver_batch`	Inputs, layers, activation functions	Performs forward propagation while saving intermediate outputs.
`backpropagation_batch`	Inputs, layers, activations, targets, derivatives, cost and regularization gradients	Computes loss gradients with respect to network parameters using backpropagation.
`optimiser_step`	Method, parameters, gradient, state, step, learning rate	Updates network parameters using the selected optimization algorithm.
`train_network`	Layers, data, activations, optimizer, learning rate, epochs, cost, regularization, validation data, batch size, seed	Trains the network using mini-batch optimization and optionally evaluates validation performance.
`fit_configuration`	Data, layer widths, activation, optimizer, learning rate, epochs, cost, regularization	Trains a network for a specified model configuration.
`refit_for_test`	Data, selected configuration	Refits the selected configuration for final test evaluation.

TABLE XII: Overview of the functions used for neural network construction, training, and evaluation.